Cravey Crust Automation Platform

Complete Technical Manual, Architectural Blueprint & Operational Handbook

Author: Cravey Systems Engineering
Document Type: System Book & Reference Guide
Version: 2.4.0 (Enterprise Edition)
Target Stack: Node.js 20+ · Baileys · PostgreSQL · Prisma · Multi-LLM

PURPOSE OF THIS MANUAL

This volume provides an exhaustive, book-style technical manual for the **Cravey Crust WhatsApp Autonomous Food Ordering & Kitchen Automation Platform**. It details the real-world motivations behind its development (*Kyun Banaya*), the complete technological stack and design patterns selected (*Kia Stack Use Kiya*), a rigorous step-by-step installation and operational guide (*Kese Use Karein*), the internal state machines governing conversational commerce, database relational schemas, REST API surfaces, and maintenance protocols.

Table of Contents

Chapter 1: The Problem Space & Business Motivation (Kyun Banaya?) Page 2
Chapter 2: Technical Stack & Architectural Decisions (Kia Stack Use Kiya?) Page 4
Chapter 3: The Hybrid Engine: Fast-Path Regex & Multi-LLM Failover Page 7
Chapter 4: End-to-End Operational Workflow & Ordering Lifecycle Page 10
Chapter 5: Step-by-Step Installation & Environment Setup Page 13
Chapter 6: WhatsApp Connection & Multi-Session Pairing Manual Page 16
Chapter 7: Kitchen & Administration Web Dashboard Manual Page 18
Chapter 8: Relational Database Schema & Prisma ORM Blueprint Page 21
Chapter 9: Complete REST API & System Probes Reference Page 24
Chapter 10: Bot Commands, Production Hardening & Troubleshooting Page 26

Chapter 1: The Problem Space & Business Motivation (Kyun Banaya?)

1.1 The High Cost and Friction of Food Aggregators

For high-volume restaurants, cloud kitchens, and emerging food franchises, customer acquisition and retention are frequently compromised by traditional third-party food delivery platforms (such as Foodpanda, UberEats, and Deliveroo). These aggregators impose commissions ranging between **22% to 35% on every single order**. For a food establishment operating on a 15% to 20% net profit margin, these commissions effectively consume all underlying profitability, forcing businesses into unsustainable pricing models.

Furthermore, aggregators intentionally blindfold the restaurant from its own customers. The restaurant receives a masked phone number, lacks insight into purchase frequency, and cannot execute direct remarketing or loyalty incentives. In essence, the restaurant functions merely as an outsourced cooking line for the aggregator's proprietary customer base.

1.2 The Failure of Proprietary Mobile Apps

In response to aggregator fees, many restaurants spend significant capital developing standalone Android and iOS mobile applications. However, customer conversion and retention on standalone apps remain abysmally low due to measurable consumer friction:

- **Download Hesitation:** Consumers are reluctant to download a 50MB-120MB app merely to order a pizza once every two weeks.
- **Authentication Barriers:** Forgotten passwords, multi-step email verifications, and mandatory profile creation result in a 65%+ cart abandonment rate at the authentication screen.
- **Storage & Notification Fatigue:** Consumers frequently delete rarely used apps or disable push notifications, negating the restaurant's marketing spend.

1.3 The WhatsApp Advantage

In contrast to proprietary apps, **WhatsApp is already installed on over 92% of all smartphones** in Pakistan, South Asia, the Middle East, and Latin America. It is opened dozens of times daily, requires no onboarding or password authentication, and enjoys an open rate of over 95%. Ordering food via WhatsApp is an organic, effortless behavior that mirrors how consumers interact with friends and family.

1.4 The Human Bottleneck in Traditional WhatsApp Ordering

Historically, restaurants attempting to take orders manually via WhatsApp run into severe operational gridlocks:

1. **Peak-Time Bottlenecks:** Between 7:30 PM and 10:30 PM (dinner rush), a restaurant may receive 80 to 200 concurrent incoming chats. Two or three human order takers cannot read, type, clarify addresses, calculate totals, and record orders fast enough. Customers experience 15-25 minute response delays and abandon their carts to competitors.
2. **Human Transcription Errors:** Manual order booking leads to misheard pizza sizes, forgotten add-ons, inaccurate manual delivery fee calculations, and misspelled customer addresses.
3. **Bilingual Communication Hurdles:** In Pakistan, customers naturally communicate in **Roman Urdu** (Urdu written in Latin letters, e.g., *"Aik large fajita pizza kr do aur 2 cold drink"*), mixed English, or Urdu script. Traditional rule-based chatbots cannot parse linguistic nuances, slang, spelling variations, or informal negotiations.

1.5 The Core Mission of Cravey Crust

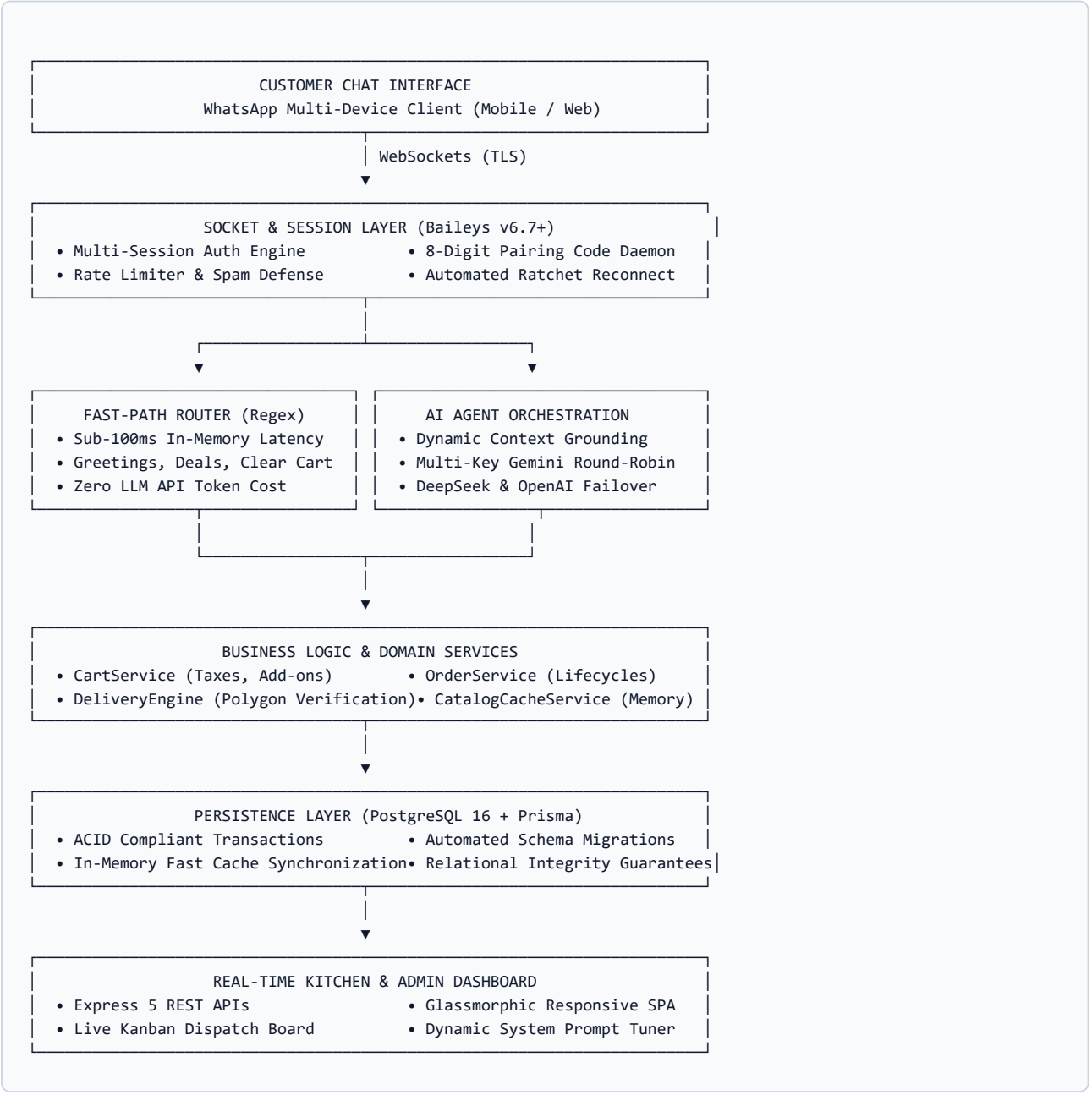
The **Cravey Crust Full Bot** was designed and engineered specifically to bridge these systemic flaws. It delivers an autonomous, high-throughput software system that:

- Operates 24/7 directly inside WhatsApp with zero human labor required for 95%+ of incoming standard orders.
- Possesses native bilingual comprehension of **Roman Urdu, English, and Urdu**, speaking with a warm, culturally resonant restaurant persona (*"Burger Raja"*).
- Responds in **sub-100 milliseconds** for standard catalog browsing and frequently asked questions via an in-memory fast-path engine.
- Guarantees **zero order omissions** through a deterministic 4-step ordering protocol (Item Selection ➜ Full Name ➜ Verified Delivery Zone ➜ Payment Confirmation).
- Provides human kitchen staff and managers with a real-time, glassmorphic visual **Kitchen Kanban Dashboard** for order dispatch and fulfillment.
- Eliminates platform commission fees completely.

Chapter 2: Technical Stack & Architectural Decisions (Kia Stack Use Kiya?)

2.1 High-Level Component Topology

The Cravey Crust platform adopts a modular, service-oriented architecture designed to ensure fault tolerance, zero-downtime menu synchronization, low latency, and deterministic order persistence.



2.2 Justification of Technology Stack

1. Node.js (v20+ LTS, ECMAScript Modules)

Node.js was selected for its single-threaded, non-blocking asynchronous event loop. In conversational platforms, the runtime is predominantly I/O-bound: managing hundreds of persistent WebSocket connections, querying databases, handling HTTP network streams to LLM APIs, and broadcasting real-time dashboard events. The native ESM structure provides strict modularity, high tree-shaking efficiency, and immediate module loading.

2. WhatsApp Engine: @whiskeysockets/baileys (v6.7+)

Unlike official Meta WhatsApp Cloud APIs, which charge substantial per-conversation conversation fees ($0.03 to $0.07 per user conversation) and enforce rigid interactive message templates that prevent natural Roman Urdu conversational flow, **Baileys connects directly to WhatsApp's multi-device WebSockets infrastructure**:

- **Zero Per-Message Cost:** The restaurant pays zero messaging fees regardless of whether they process 100 or 10,000 orders monthly.
- **8-Digit Pairing Code Support:** Eliminates the traditional need to screenshot a QR code with a secondary phone. Staff simply enter their phone number on the dashboard and receive an 8-digit link code natively inside WhatsApp.
- **Cryptographic Session Storage:** Stores authentication ratchets, identity keys, and pre-keys securely inside the local file system (`auth/`), allowing instantaneous auto-reconnects on system reboot.

3. Database Layer: PostgreSQL 16 & Prisma ORM

Food orders require strict relational consistency, atomicity, and financial accuracy. A cart item must reference valid menu variants, modifiers, and real-time delivery zone pricing.

- **PostgreSQL 16:** Offers robust ACID guarantees, jsonb support for arbitrary customer metadata, and rock-solid transaction isolation levels that prevent double-booking.
- **Prisma ORM:** Provides compile-time type safety, automated migration generators (`prisma migrate`), declarative relational definitions in `schema.prisma`, and a powerful schema management GUI (`npx prisma studio`).

4. In-Memory Catalog Cache (catalogCacheService.js)

During peak traffic, hundreds of users browse pizza deals and menu categories concurrently. Querying the relational database for static menu items on every keystroke induces unnecessary database I/O load. `catalogCacheService` maintains an in-memory synchronized clone of the active catalog, variants, and deals, reducing menu read latency from 45ms (database query) to **less than 2ms** (RAM lookup).

5. Real-Time Admin Frontend: Express 5 + Modern Glassmorphic SPA

The kitchen and admin dashboard (served from `/public`) was built deliberately using **Vanilla ES6+ JavaScript, semantic HTML5, and native CSS3 Glassmorphism** rather than complex frontend frameworks (Next.js, React, or Angular):

- **Zero Build Latency:** No Webpack, Vite, or npm build steps required on the production server. Changes are immediately hot-served.

- **Minimal Footprint:** Runs seamlessly on budget 1GB-RAM Linux VPS instances or local kitchen POS computers without consuming excessive CPU or RAM.
- **Cross-Device Compatibility:** Perfectly responsive on low-cost Android kitchen tablets, desktop monitors, and iPhones.

Chapter 3: The Hybrid Engine: Fast-Path Regex & Multi-LLM Failover

3.1 The Deterministic Fast-Path Engine (Sub-100ms)

A common anti-pattern in AI chatbots is routing every single user utterance directly to an external LLM API. This introduces 1,500ms to 3,500ms of network latency, exhausts third-party API quotas, and introduces non-deterministic hallucinations into basic conversational flows.

Cravey Crust implements an upfront **Deterministic Intent Parser (`src/services/intentParser.js`)**. Before invoking any external AI engine, incoming customer messages are evaluated against normalized regex matrices:

Customer Utterance (Example)	Matched Fast-Path Intent	Engine Execution	Response Latency
"Salam", "Hi", "Hello", "Menu dikhao"	`INTENT_GREETING_MENU`	Fetches top categories from `catalogCacheService` & outputs formatted greeting.	**~40ms**
"Pizza deals", "Deals kya hain"	`INTENT_CATEGORY_DEALS`	Filters cache for active bundle deals and prints formatted pricing list.	**~55ms**
"Cart clear kr do", "Order cancel"	`INTENT_CLEAR_CART`	Deletes customer's active pending cart in Prisma and confirms reset.	**~70ms**
"Store timing kya hai", "Location kahan hai"	`INTENT_STORE_INFO`	Reads operational hours and physical address directly from cache.	**~35ms**
"Mujhe agent se baat karni hai", ".handover"	`INTENT_HANDOVER`	Sets `isHandoverActive: true`, silences bot, and notifies kitchen dashboard.	**~60ms**

3.2 Dynamic Context Grounding for Complex Queries

When an utterance requires semantic reasoning (e.g., *"Hum 4 dost hain, koi aisi deal batao jisme 2 pizza aur wings hon aur Rs. 3000 se kam ho"*), the message bypasses the fast-path engine and is routed to the **AI Agent Service (`src/services/aiAgentService.js`)**.

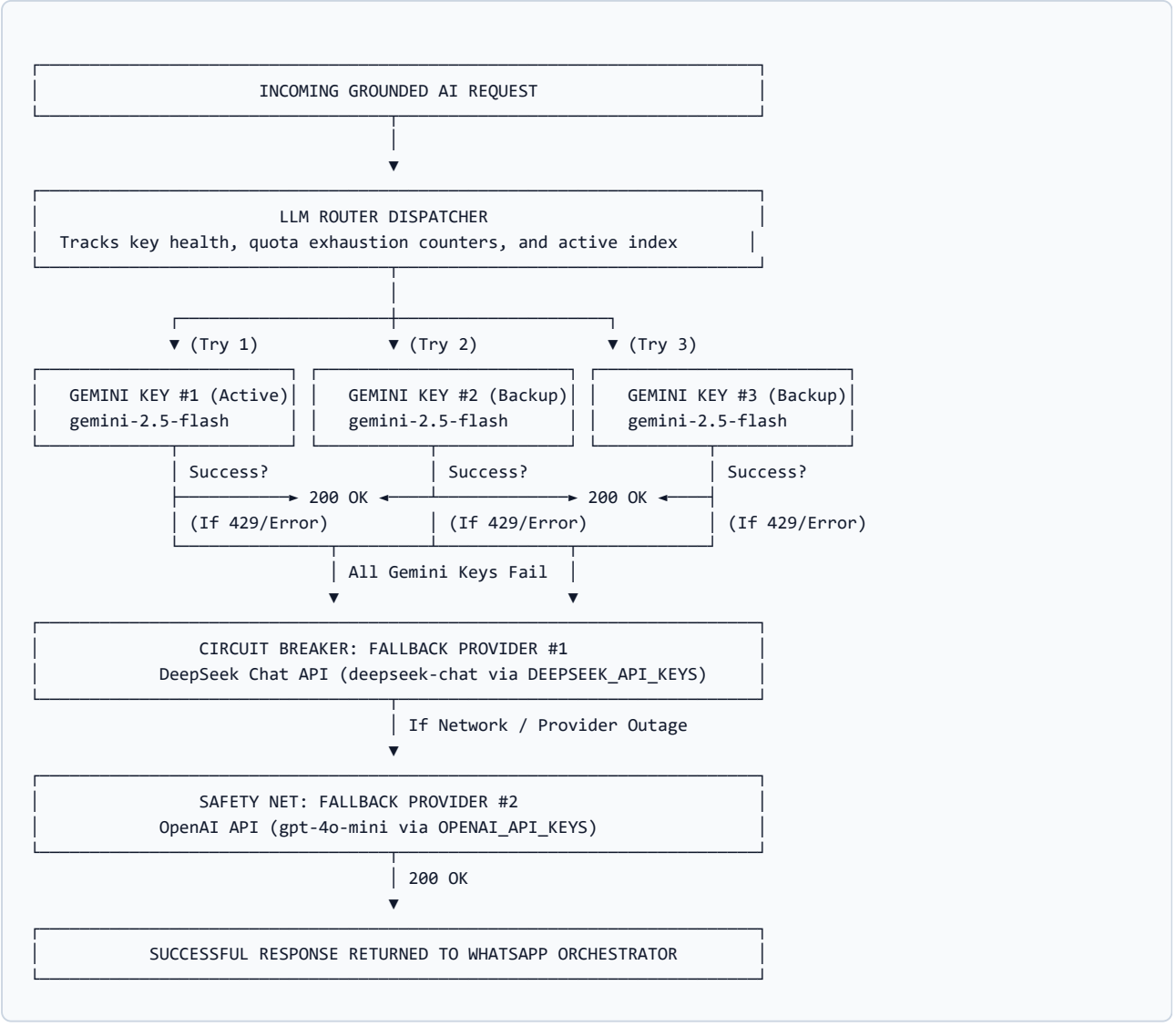
To ensure the LLM never hallucinates items not currently in stock or invents fictional prices, the orchestrator executes **Real-Time Dynamic Prompt Grounding**:

1. The customer's current cart state, item quantities, and subtotal are injected into the system prompt.
2. The active menu catalog (with real prices, available sizes, and descriptions) is summarized and appended.
3. Active store operational rules (minimum order limits, delivery areas) are appended.

4. The model is strictly instructed to return both a natural conversational response in Roman Urdu/English and an optional structured JSON payload instructing the cart service to add, modify, or delete line items.

3.3 Multi-Key & Multi-Provider Round-Robin Failover Architecture

The single greatest operational hazard for an AI-driven restaurant bot is a `HTTP 429 Resource Exhausted` or rate-limit error during the dinner rush. Cravey Crust solves this through an enterprise **Multi-Key Round-Robin & Fallback Router (`src/services/llmRouter.js`)**.



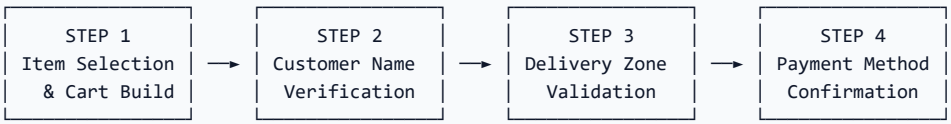
Key Rotation Details:

- `GEMINI_API_KEYS` accepts a comma-separated list of keys in `.env` (e.g. `AIzaSyA...,AIzaSyB...,AIzaSyC...`).
- Keys rotate on a round-robin index. If Key #1 encounters a rate limit (429) or transient error (503), it is marked in a temporary cooldown window and Key #2 is immediately tried with zero delay to the customer.
- If all configured Gemini keys fail, the router triggers a circuit break and routes the payload to `AI_FALLBACK_PROVIDER` (DeepSeek Chat), followed by OpenAI.

Chapter 4: End-to-End Operational Workflow & Ordering Lifecycle

4.1 The 4-Step Checkout Protocol

To prevent abandoned, vague, or unserviceable orders, Cravey Crust implements a deterministic 4-step state machine. The bot will never commit an order to the kitchen without strictly validating all four criteria:



Step 1: Item Selection & Cart Assembly

The customer selects one or more items (e.g. *"1 Large Fajita Pizza and 1 Garlic Bread"*). The item names are matched against the database catalog. The bot confirms the addition, provides the running subtotal, and automatically inquires about crust preference or drinks if applicable.

Step 2: Customer Name Verification

If the customer's phone number is already registered in the PostgreSQL `Customer` table from a previous order, their name is pre-populated (e.g., *"Asim bhai, aapka naam Asim Khan hi use karein?"*). For new users, the bot systematically requests their full name to attach to the kitchen ticket.

Step 3: Delivery Zone Verification & Fee Calculation

The customer shares their street address, sector, or sends a native WhatsApp Live Location. The **Delivery Verification Engine** compares the input against active records in `DeliveryArea`:

- If the address falls within an approved zone (e.g., *"Sector F-8"*), the zone's flat delivery charge (e.g., Rs. 150) is calculated and added to the order bill.
- If the total order amount does not satisfy the zone's `minOrderAmount`, the bot politely notifies the user how much more they need to add to qualify.
- If the location is out of bounds, the customer is gently informed that delivery is unavailable and offered self-pickup instead.

Step 4: Payment Selection & Kitchen Commitment

The bot presents the final itemized receipt and requests payment method selection:

1. **Cash on Delivery (COD)**
2. **JazzCash** (Displays restaurant merchant tilt ID / account number)
3. **EasyPaisa** (Displays account details and requests transaction ID / screenshot)

Upon confirmation, the status updates from `CART` to `PENDING` in the database. A real-time notification rings on the kitchen dashboard, and a confirmation ticket with an estimated delivery window (35-45 mins) is sent to the customer.

4.2 Human Handover & Staff Intervention Protocol

Automation should never alienate customers requiring custom assistance. Cravey Crust incorporates an instantaneous handover protocol:

- **Customer Trigger:** If a customer types `.handover` or phrases like *"Mujhe kisi insan ya manager se baat karni hai"*, the bot sets `isHandoverActive = true` for that specific customer record.
- **Dashboard Takeover:** Kitchen staff can click **"Takeover Chat"** directly on the web dashboard.
- **AI Suppression:** While handover is active, the AI engine completely suppresses its automated replies for that user. Human staff can chat with the customer normally through the physical WhatsApp phone or web dashboard.
- **Resume Automation:** Once the inquiry is resolved, staff can click **"Resume AI"** on the dashboard or send `.takeover` from the admin phone to hand control back to the bot.

Chapter 5: Step-by-Step Installation & Environment Setup

5.1 Prerequisites

Before installing the platform, ensure the hosting environment meets the following specifications:

Component	Minimum Specification	Recommended Specification
Operating System	Ubuntu 20.04 LTS / Windows 10	Ubuntu 22.04 / 24.04 LTS (64-bit)
Node.js Runtime	v20.0.0	v20.18.0 LTS or v22.x LTS
Database	PostgreSQL 14	PostgreSQL 16 (Local or Managed RDS)
RAM / CPU	1 GB RAM / 1 vCPU	2 GB - 4 GB RAM / 2 vCPUs
System Dependencies	Git, OpenSSL	Git, OpenSSL, FFmpeg (for media/stickers)

5.2 Option A: Local / Native Linux & Windows Installation

Step 1: Clone Repository

```
git clone https://github.com/workwithasim/full-bot.git
cd full-bot
```

Step 2: Install Node Dependencies

```
npm install
```

Step 3: Configure Environment Variables

```
cp .env.example .env
```

Open `.env` in your editor and configure the essential parameters:

```
PORT=3000
SERVER_IP=localhost
PREFIX=.
OWNER_NUMBER=923001234567

# Admin Dashboard Security
DASHBOARD_USER=admin
DASHBOARD_PASS=your_secure_password_here

# PostgreSQL Database Connection URI
DATABASE_URL="postgresql://postgres:password@localhost:5432/cravey_crust?schema=public"

# AI Configuration (Comma-separated keys for auto-rotation)
STANDALONE_AI_ENABLED=true
AI_PRIMARY_PROVIDER=gemini
AI_FALLBACK_PROVIDER=deepseek
GEMINI_API_KEYS="AIzaSyKey1,AIzaSyKey2,AIzaSyKey3"
GEMINI_MODEL="gemini-2.5-flash"
```

Step 4: Execute Database Migrations & Initial Data Seeding

```
# Push schema models to PostgreSQL
npx prisma db push

# Generate Prisma Client
npm run db:generate

# Populate database with initial pizza deals, categories, and delivery areas
npm run db:seed
```

Step 5: Start Application

```
# For development (auto-restarts on file edit)
npm run dev

# Or directly with Node
node bot.js
```

5.3 Option B: One-Click Production Deployment with Docker Compose

The repository includes a production-grade `docker-compose.yml` file that automatically provisions the Node.js application, an isolated PostgreSQL 16 database container, and persistent host volumes.

```
# 1. Clone repository
git clone https://github.com/workwithasim/full-bot.git
cd full-bot

# 2. Setup your .env file
cp .env.example .env

# 3. Build and launch services in background
docker compose up -d --build

# 4. View live logs from the bot container
docker compose logs -f app
```

To safely shut down the containers while preserving all order history and sessions:

```
docker compose down
```

5.4 Option C: Process Supervision with PM2 (Bare-Metal VPS)

When running directly on an Ubuntu/Debian VPS without Docker, use **PM2** to ensure the application restarts automatically upon server reboot or unexpected exceptions:

```
# 1. Install PM2 globally
npm install -g pm2

# 2. Launch the application
pm2 start bot.js --name "cravey-bot"

# 3. Configure PM2 systemd daemon to survive server reboots
pm2 startup
pm2 save
```

Chapter 6: WhatsApp Connection & Multi-Session Pairing Manual

6.1 Overview of Connection Methods

Cravey Crust provides two independent pairing protocols through its modern web dashboard (accessible at `http://localhost:3000`).

6.2 Method 1: The 8-Digit Pairing Code (Recommended)

The pairing code method is the fastest, cleanest, and most reliable way to link a restaurant phone number. It requires only one device and does not involve pointing a camera at a screen.

```
Step 1: Open Dashboard (http://localhost:3000)
Step 2: Enter WhatsApp Number (e.g. 923001234567) ──► Click "Get Pairing Code"
Step 3: Server generates 8-digit alphanumeric token (e.g. "K7WP-92LA")
Step 4: On Phone: WhatsApp ➔ Linked Devices ➔ Link a Device ➔ "Link with phone number instead"
Step 5: Enter the 8-digit code ──► Immediate Secure Pairing & Session Persistence
```

1. Access the dashboard at `http://localhost:3000`.
2. In the **WhatsApp Connection Hub** card, select **"Pairing Code Mode"**.
3. Enter the phone number hosting the restaurant WhatsApp account, including country code (e.g., `923001234567` for Pakistan). Do not enter `+` or dashes.
4. Click **Request Pairing Code**. Within 2-3 seconds, an 8-character code will appear in large letters.
5. Open WhatsApp on the target smartphone. Tap the three dots (Android) or Settings (iPhone) **➔ Linked Devices ➔ Link a Device**.
6. Tap **"Link with phone number instead"** at the bottom of the camera screen.
7. Enter the 8 characters. The socket will authenticate immediately, and the terminal will output: `[WhatsApp] Connection established successfully!`

6.3 Method 2: High-Resolution QR Code Scanner

If you prefer traditional QR pairing:

1. Select **"QR Code Mode"** on the dashboard or watch your server terminal.
2. A high-resolution QR code will render both in the web browser and directly inside the terminal window via ASCII graphics.
3. Open WhatsApp **➔ Linked Devices ➔ Link a Device ➔** Point the camera at the code.

6.4 Session Security & Persistence Mechanics

> **CRITICAL SECURITY NOTICE**
>
> All encryption keys, ratchet tokens, and device credentials are saved inside the local `auth/` directory. This directory is strictly included in `.gitignore`. **Never copy, zip, or commit the `auth/` directory to public version control**, as anyone with access to these files can impersonate your WhatsApp account.

The platform features an automated **Socket Reconnection & Ratchet Healing Daemon**. If the server loses internet connectivity or restarts, it re-reads the credentials from `auth/` and restores connection without requiring staff to re-scan or enter codes again.

Chapter 7: Kitchen & Administration Web Dashboard Manual

7.1 Accessing the Dashboard

Open your web browser and navigate to:

```
http://<your-server-ip>:3000
```

Log in with the credentials defined in your `.env` file (Default: User: `admin`, Password: `xortlogix`).

7.2 The Kitchen Kanban Pipeline

The order fulfillment workflow is visualized through an interactive Kanban board designed for kitchen display systems (KDS):

Pipeline Stage	Database Status	Kitchen Action Required	Automated Customer WhatsApp Trigger
1. New Incoming	`PENDING`	Chef reviews items; clicks **"Accept Order"**.	*"Aapka order kitchen ne accept kar liya hai! Preparing..."*
2. In Kitchen	`PREPARING`	Ovens & fryers working; items being packed into boxes.	Customer can check live countdown via `.status`.
3. Out for Delivery	`OUT_FOR_DELIVERY`	Handed to dispatch rider; rider phone number attached.	*"Aapka order rider ko hand over ho chuka hai! On the way."*
4. Fulfilled	`DELIVERED`	Rider confirms payment received & order handed over.	*"Mubarak ho! Order deliver ho chuka hai. Feedback dein!"*
5. Cancelled	`CANCELLED`	Item out of stock or fake customer details detected.	*"Maazrat! Aapka order cancel kar diya gaya hai."*

7.3 Catalog, Variants & Deal Management

Under the **"Menu & Deals"** tab, restaurant managers can make live menu adjustments:

- **Instant Stock Toggle:** Click on any pizza, burger, or beverage to toggle its `isAvailable` flag. The in-memory cache updates instantly. If an item is disabled, the AI agent will immediately refuse to sell it on WhatsApp.
- **Variant Matrices:** Define sizes (Small, Regular, Large, Jumbo) and crust types (Thin Crust, Stuffed Crust, Pan Crust) with respective price increments.

- **Combo Deals Engine:** Create deals with bundled rules (e.g., *"Deal 1 = 1 Large Pizza + 2 Drinks for Rs. 1450"*).

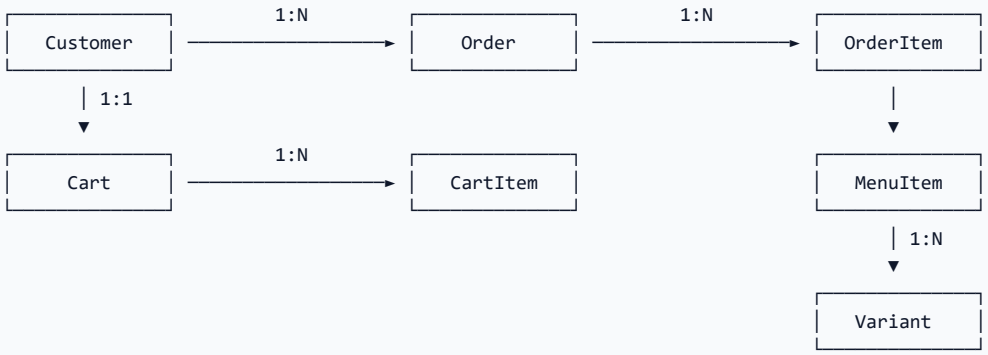
7.4 Live AI Prompt & Persona Tuner

Under the **"AI Settings"** tab, managers can customize the bot's behavior in real time:

- **Personality Tuning:** Modify the tone from friendly Pakistani Roman Urdu (*"Burger Raja"*) to formal English.
- **Special Announcement Injections:** Add temporary rules (e.g., *"Aj barish ki wajah se delivery mein 15 minute extra lag sakte hain"*). The AI will immediately integrate this notice into conversational replies without restarting the Node.js server.

Chapter 8: Relational Database Schema & Prisma ORM Blueprint

8.1 Entity Relationship Diagram (ERD) Overview



8.2 Core Prisma Data Models (prisma/schema.prisma)

1. Order & OrderItem Entities

```
model Order {
  id              String        @id @default(uuid())
  orderNumber     String        @unique // e.g. "CRV-8942"
  customerId      String
  customer        Customer      @relation(fields: [customerId], references: [id])
  status          OrderStatus   @default(PENDING) // PENDING, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCEL
  totalAmount     Decimal       @db.Decimal(10, 2)
  deliveryFee     Decimal       @default(0.00) @db.Decimal(10, 2)
  deliveryAddress String
  deliveryAreaId  String?
  deliveryArea    DeliveryArea? @relation(fields: [deliveryAreaId], references: [id])
  paymentMethod   PaymentMethod @default(COD) // COD, JAZZCASH, EASYPAISA
  paymentStatus   PaymentStatus @default(UNPAID)
  items           OrderItem[]
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt
}

model OrderItem {
  id          String    @id @default(uuid())
  orderId     String
  order       Order     @relation(fields: [orderId], references: [id], onDelete: Cascade)
  menuItemId  String
  menuItem    MenuItem  @relation(fields: [menuItemId], references: [id])
  variantId   String?
  variant     Variant?  @relation(fields: [variantId], references: [id])
  quantity    Int       @default(1)
  unitPrice   Decimal   @db.Decimal(10, 2)
  totalPrice  Decimal   @db.Decimal(10, 2)
}
```

2. Customer & In-Flight Cart Entities

```
model Customer {
  id            String    @id @default(uuid())
  phoneNumber   String    @unique // WhatsApp Phone with country code
  fullName      String?
  savedAddress  String?
  isHandover    Boolean   @default(false)
  cart          Cart?
  orders        Order[]
  createdAt     DateTime  @default(now())
}

model Cart {
  id          String     @id @default(uuid())
  customerId  String     @unique
  customer    Customer   @relation(fields: [customerId], references: [id], onDelete: Cascade)
  items       CartItem[]
  updatedAt   DateTime   @updatedAt
}
```

Chapter 9: Complete REST API & System Probes Reference

9.1 Authentication & Request Headers

Administrative endpoints located under `/api/admin/*` require an authenticated session or an authorization bearer token matching the configured secret.

9.2 Key Administrative Endpoints

HTTP Method	Endpoint Route	Description	Sample Response / Status
`GET`	`/api/admin/orders`	Lists all active orders with optional `?status=PENDING` filter.	`200 OK [{"id":"uuid","orderNumber":"CRV-101",...}]`
`PATCH`	`/api/admin/orders/:id/status`	Updates order state & triggers customer WhatsApp alert.	`200 OK {"success":true,"newStatus":"PREPARING"}`
`GET`	`/api/admin/menu`	Fetches complete catalog with categories & pricing variants.	`200 OK [{"id":"m1","name":"Fajita Pizza",...}]`
`POST`	`/api/admin/menu`	Creates a new pizza, burger, or beverage item.	`201 Created {"success":true,"itemId":"uuid"}`
`GET`	`/api/admin/deals`	Retrieves active combo bundle deals.	`200 OK [{"dealName":"Deal 1","price":750,...}]`
`GET`	`/api/admin/prompt`	Returns current AI persona system prompt.	`200 OK {"systemPrompt":"You are Burger Raja..."}`
`POST`	`/api/admin/prompt`	Hot-reloads AI prompt with zero server downtime.	`200 OK {"success":true,"message":"Prompt reloaded"}`
`GET`	`/health`	Liveness probe checking DB, memory & WhatsApp socket.	`200 OK {"status":"healthy","db":"connected"}`

Chapter 10: Bot Commands, Production Hardening & Troubleshooting

10.1 WhatsApp Bot Commands Reference

Command	Permission	Functional Behavior
`.menu`	Public (All)	Prints the restaurant's complete categorized menu directly in chat.
`.status`	Public (All)	Queries the customer's most recent active order and prints real-time status.
`.handover`	Public (All)	Pauses the AI agent and alerts human kitchen staff that assistance is requested.
`.takeover`	Admin / Owner	Unpauses the AI agent and resumes automated order handling.
`.ping`	Admin / Owner	Measures bot socket round-trip latency and prints current memory consumption.
`.mute` / `.unmute`	Admin	Temporarily silences bot activity in a designated chat or group.
`.sticker`	Public (All)	Converts an attached photo or 7-second video into a WhatsApp sticker via FFmpeg.
`.vv`	Owner Only	Extracts and recovers view-once (single-view) photos and videos for audit.

10.2 Production Hardening & Operational Best Practices

1. **SSL Termination with Nginx Reverse Proxy:** Never expose port 3000 directly to the public internet. Deploy Nginx in front of the application with Let's Encrypt SSL certificates.
2. **Automated Database Backups:** Schedule a daily cron job executing `pg_dump` to back up customer order history to offsite storage:

   ```
   0 3 * * * pg_dump -U cravey cravey_crust > /backups/cravey_$(date +\%F).sql
   ```

3. **WhatsApp Account Warmup:** If linking a brand-new SIM card or WhatsApp account, do not blast 500 promotional broadcast messages on day one. Allow the account to warm up organically over 5-7 days with normal conversations to prevent WhatsApp automated anti-spam flags.

10.3 Troubleshooting & Frequently Encountered Issues

ISSUE 1: Port 3000 Already In Use (EADDRINUSE)

Cause: A previous Node.js instance or another service is occupying port 3000.

Remedy: On Linux: `lsof -i :3000 | awk 'NR>1 {print $2}' | xargs kill -9`.
On Windows: `Get-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess | Stop-Process`, or modify `PORT=3001` in `.env`.

ISSUE 2: WhatsApp Session Disconnected / Logged Out

Cause: The host phone remained disconnected from the internet for over 14 days, or the user manually selected "Log out" from Linked Devices.

Remedy: Delete the stale authentication folder: `rm -rf auth/` and re-pair using the 8-digit pairing code on the web dashboard.

ISSUE 3: AI Quota Rate Limit (HTTP 429)

Cause: High traffic volume exceeded the free-tier requests per minute (RPM) for a single Gemini API key.

Remedy: Add 2-3 additional Gemini API keys in `.env` under `GEMINI_API_KEYS="key1,key2,key3"` or configure `DEEPSEEK_API_KEYS` as fallback.
